二、异常日志

(一) 错误码

1. 【强制】错误码的制定原则：快速溯源、沟通标准化。
   说明：错误码想得过于完美和复杂，就像康熙字典的生僻字一样，用词似乎精准，但是字典不容易随身携带且简单易懂。
   正例：错误码回答的问题是谁的错？错在哪？
   1）错误码必须能够快速知晓错误来源，可快速判断是谁的问题。
   2）错误码必须能够进行清晰地比对（代码中容易 equals）。
   3）错误码有利于团队快速对错误原因达到一致认知。

2. 【强制】错误码不体现版本号和错误等级信息。
   说明：错误码以不断追加的方式进行兼容。错误等级由日志和错误码本身的释义来决定。

3. 【强制】全部正常，但不得不填充错误码时返回五个零：00000。

4. 【强制】错误码为字符串类型，共 5 位，分成两个部分：错误产生来源+四位数字编号。
   说明：错误产生来源分为 A/B/C，A 表示错误来源于用户，比如参数错误，用户安装版本过低，用户支付超时等问题；B 表示错误来源于当前系统，往往是业务逻辑出错，或程序健壮性差等问题；C 表示错误来源于第三方服务，比如 CDN 服务出错，消息投递超时等问题；四位数字编号从 0001 到 9999，大类之间的步长间距预留 100，参考文末附表 3。

5. 【强制】编号不与公司业务架构，更不与组织架构挂钩，以先到先得的原则在统一平台上进行，审批生效，编号即被永久固定。

6. 【强制】错误码使用者避免随意定义新的错误码。
   说明：尽可能在原有错误码附表中找到语义相同或者相近的错误码在代码中使用即可。

7. 【强制】错误码不能直接输出给用户作为提示信息使用。
   说明：堆栈（stack_trace）、错误信息(error_message)、错误码（error_code）、提示信息（user_tip）是一个有效关联并互相转义的和谐整体，但是请勿互相越俎代庖。

8. 【推荐】错误码之外的业务信息由 error_message 来承载，而不是让错误码本身涵盖过多具体业务属性。

9. 【推荐】在获取第三方服务错误码时，向上抛出允许本系统转义，由 C 转为 B，并且在错误信息上带上原有的第三方错误码。

10. 【参考】错误码分为一级宏观错误码、二级宏观错误码、三级宏观错误码。
    说明：在无法更加具体确定的错误场景中，可以直接使用一级宏观错误码，分别是：A0001（用户端错误）、B0001（系统执行出错）、C0001（调用第三方服务出错）。
    正例：调用第三方服务出错是一级，中间件错误是二级，消息服务出错是三级。

11. 【参考】错误码的后三位编号与 HTTP 状态码没有任何关系。

12. 【参考】错误码有利于不同文化背景的开发者进行交流与代码协作。
    说明：英文单词形式的错误码不利于非英语母语国家（如阿拉伯语、希伯来语、俄罗斯语等）之间的开发者互相协作。

13. 【参考】错误码即人性，感性认知+口口相传，使用纯数字来进行错误码编排不利于感性记忆和分类。
    说明：数字是一个整体，每位数字的地位和含义是相同的。
    反例：一个五位数字 12345，第 1 位是错误等级，第 2 位是错误来源，345 是编号，人的大脑不会主动地拆开并分辨每位数字的不同含义。

(二) 异常处理

1. 【强制】Java 类库中定义的可以通过预检查方式规避的 RuntimeException 异常不应该通过 catch 的方式来处理，比如：NullPointerException，IndexOutOfBoundsException 等等。
   说明：无法通过预检查的异常除外，比如，在解析字符串形式的数字时，可能存在数字格式错误，不得不通过 catch NumberFormatException 来实现。
   正例：if (obj != null) {...}
   反例：try { obj.method(); } catch (NullPointerException e) {...}

2. 【强制】异常捕获后不要用来做流程控制，条件控制。
   说明：异常设计的初衷是解决程序运行中的各种意外情况，且异常的处理效率比条件判断方式要低很多。

3. 【强制】catch 时请分清稳定代码和非稳定代码，稳定代码指的是无论如何不会出错的代码。对于非稳定代码的 catch 尽可能进行区分异常类型，再做对应的异常处理。
   说明：对大段代码进行 try-catch，使程序无法根据不同的异常做出正确的应激反应，也不利于定位问题，这是一种不负责任的表现。
   正例：用户注册的场景中，如果用户输入非法字符，或用户名称已存在，或用户输入密码过于简单，在程序上作出分门别类的判断，并提示给用户。

4. 【强制】捕获异常是为了处理它，不要捕获了却什么都不处理而抛弃之，如果不想处理它，请将该异常抛给它的调用者。最外层的业务使用者，必须处理异常，将其转化为用户可以理解的内容。

5. 【强制】事务场景中，抛出异常被 catch 后，如果需要回滚，一定要注意手动回滚事务。

6. 【强制】finally 块必须对资源对象、流对象进行关闭，有异常也要做 try-catch。
   说明：如果 JDK7，可以使用 try-with-resources 方式。

7. 【强制】不要在 finally 块中使用 return
   说明：try 块中的 return 语句执行成功后，并不马上返回，而是继续执行 finally 块中的语句，如果此处存在 return 语句，则会在此直接返回，无情丢弃掉 try 块中的返回点。
   反例：

```
private int x = 0;
public int checkReturn() {
    try {
        // x 等于 1，此处不返回
        return ++x;
    } finally {
        // 返回的结果是 2
        return ++x;
    }
}
```

8. 【强制】捕获异常与抛异常，必须是完全匹配，或者捕获异常是抛异常的父类。
   说明：如果预期对方抛的是绣球，实际接到的是铅球，就会产生意外情况。

9. 【强制】在调用 RPC、二方包、或动态生成类的相关方法时，捕捉异常使用 Throwable 类进行拦截。
   说明：通过反射机制来调用方法，如果找不到方法，抛出 NoSuchMethodException。什么情况会抛出 NoSuchMethodError 呢？二方包在类冲突时，仲裁机制可能导致引入非预期的版本使类的方法签名不匹配，或者在字节码修改框架（比如：ASM）动态创建或修改类时，修改了相应的方法签名。这些情况，即使代码编译期是正确的，但在代码运行期时，会抛出 NoSuchMethodError。
   反例：足迹服务引入了高版本的 spring，导致运行到某段核心逻辑时，抛出 NoSuchMethodError 错误，catch 用的类却是 Exception，堆栈向上抛，影响到上层业务。这是一个非核心功能点影响到核心应用的典型反例。

10. 【推荐】方法的返回值可以为 null，不强制返回空集合，或者空对象等，必须添加注释充分说明什么情况下会返回 null 值。
说明：本规约明确防止 NPE 是调用者的责任。即使被调用方法返回空集合或者空对象，对调用者来说，也并非高枕无忧，必须考虑到远程调用失败，运行时异常等场景返回 null 的情况。

11. 【推荐】防止 NPE，是程序员的基本修养，注意 NPE 产生的场景：
1）返回类型为基本数据类型，return 包装数据类型的对象时，自动拆箱有可能产生 NPE
反例：public int method() { return Integer 对象; }，如果为 null，自动解箱抛 NPE。
2）数据库的查询结果可能为 null。
3）集合里的元素即使 isNotEmpty，取出的数据元素也可能为 null。
4）远程调用返回对象时，一律要求进行空指针判断，防止 NPE。
5）对于 Session 中获取的数据，建议进行 NPE 检查，避免空指针。
6）级联调用 obj.getA().getB().getC()；一连串调用，易产生 NPE。
正例：使用 JDK8 的 Optional 类来防止 NPE 问题。

12. 【推荐】定义时区分 unchecked / checked 异常，避免直接抛出 new RuntimeException()，更不允许抛出 Exception 或者 Throwable，应使用有业务含义的自定义异常。推荐业界已定义过的自定义异常，如：DAOException / ServiceException 等。

13. 【参考】对于公司外的 http / api 开放接口必须使用错误码，而应用内部推荐异常抛出；跨应用间 RPC 调用优先考虑使用 Result 方式，封装 isSuccess() 方法、错误码、错误简短信息；应用内部推荐异常抛出。
说明：关于 RPC 方法返回方式使用 Result 方式的理由：
1）使用抛异常返回方式，调用方如果没有捕获到就会产生运行时错误。
2）如果不加栈信息，只是 new 自定义异常，加入自己的理解的 error message，对于调用端解决问题的帮助不会太多。如果加了栈信息，在频繁调用出错的情况下，数据序列化和传输的性能损耗也是问题。

(三) 日志规约

1. 【强制】应用中不可直接使用日志系统（Log4j、Logback）中的 API，而应依赖使用日志框架（SLF4J、JCL—Jakarta Commons Logging）中的 API，使用门面模式的日志框架，有利于维护和各个类的日志处理方式统一。
说明：日志框架（SLF4J、JCL--Jakarta Commons Logging）的使用方式（推荐使用 SLF4J）
使用 SLF4J：
```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
private static final Logger logger = LoggerFactory.getLogger(Test.class);
```
使用 JCL：
```
import org.apache.commons.logging.Log;
import org.apache.commons.logging.LogFactory;
private static final Log log = LogFactory.getLog(Test.class);
```

2. 【强制】日志文件至少保存 15 天，因为有些异常具备以“周”为频次发生的特点。对于当天日志，以“应用名.log”来保存，保存在/{统一目录}/{应用名}/logs/目录下，过往日志格式为：{logname}.log.{保存日期}，日期格式：yyyy-MM-dd
正例：以 mppserver 应用为例，日志保存/home/admin/mppserver/logs/mppserver.log，历史日志名称为 mppserver.log.2021-11-28

3. 【强制】根据国家法律，网络运行状态、网络安全事件、个人敏感信息操作等相关记录，留存的日志不少于六个月，并且进行网络多机备份。

4. 【强制】应用中的扩展日志（如打点、临时监控、访问日志等）命名方式：appName_logType_logName.log。logType：日志类型，如 stats / monitor / access 等；logName：日志描述。这种命名的好处：通过文件名就可知道日志文件属于什么应用，什么类型，什么目的，也有利于归类查找。
   说明：推荐对日志进行分类，将错误日志和业务日志分开放，便于开发人员查看，也便于通过日志对系统进行及时监控。
   正例：mppserver 应用中单独监控时区转换异常，如：mppserver_monitor_timeZoneConvert.log

5. 【强制】在日志输出时，字符串变量之间的拼接使用占位符的方式。
   说明：因为 String 字符串的拼接会使用 StringBuilder 的 append() 方式，有一定的性能损耗。使用占位符仅是替换动作，可以有效提升性能。
   正例：logger.debug("Processing trade with id : {} and symbol : {}", id, symbol);

6. 【强制】对于 trace / debug / info 级别的日志输出，必须进行日志级别的开关判断：
   说明：虽然在 debug(参数) 的方法体内第一行代码 isDisabled(Level.DEBUG_INT) 为真时（Slf4j 的常见实现 Log4j 和 Logback），就直接 return，但是参数可能会进行字符串拼接运算。此外，如果 debug(getName()) 这种参数内有 getName() 方法调用，无谓浪费方法调用的开销。
   正例：

```
// 如果判断为真，那么可以输出 trace 和 debug 级别的日志
if (logger.isDebugEnabled()) {
    logger.debug("Current ID is: {} and name is: {}", id, getName());
}
```

7. 【强制】避免重复打印日志，浪费磁盘空间，务必在日志配置文件中设置 additivity=false
   正例：<logger name="com.taobao.dubbo.config" additivity="false">

8. 【强制】生产环境禁止使用 System.out 或 System.err 输出或使用 e.printStackTrace() 打印异常堆栈。
   说明：标准日志输出与标准错误输出文件每次 Jboss 重启时才滚动，如果大量输出送往这两个文件，容易造成文件大小超过操作系统大小限制。

9. 【强制】异常信息应该包括两类信息：案发现场信息和异常堆栈信息。如果不处理，那么通过关键字 throws 往上抛出。
   正例：logger.error("inputParams: {} and errorMessage: {}", 各类参数或者对象 toString(), e.getMessage(), e);

10. 【强制】日志打印时禁止直接用 JSON 工具将对象转换成 String。
    说明：如果对象里某些 get 方法被覆写，存在抛出异常的情况，则可能会因为打印日志而影响正常业务流程的执行。
    正例：打印日志时仅打印出业务相关属性值或者调用其对象的 toString() 方法。

11. 【推荐】谨慎地记录日志。生产环境禁止输出 debug 日志；有选择地输出 info 日志；如果使用 warn 来记录刚上线时的业务行为信息，一定要注意日志输出量的问题，避免把服务器磁盘撑爆，并记得及时删除这些观察日志。
    说明：大量地输出无效日志，不利于系统性能提升，也不利于快速定位错误点。记录日志时请思考：这些日志真的有人看吗？看到这条日志你能做什么？能不能给问题排查带来好处？

12. 【推荐】可以使用 warn 日志级别来记录用户输入参数错误的情况，避免用户投诉时，无所适从。如非必要，请不要在此场景打出 error 级别，避免频繁报警。
    说明：注意日志输出的级别，error 级别只记录系统逻辑出错、异常或者重要的错误信息。

13. 【推荐】尽量用英文来描述日志错误信息，如果日志中的错误信息用英文描述不清楚的话使用中文描述即可，否则容易产生歧义。

说明：国际化团队或海外部署的服务器由于字符集问题，使用全英文来注释和描述日志错误信息。

14. 【推荐】为了保护用户隐私，日志文件中的用户敏感信息需要进行脱敏处理。

说明：日志排查问题时，推荐使用订单号、UUID 之类的唯一编号进行查询。